

1. ET-Minion Core

1.1. Core Overview

1.1.1. Block Partitioning

Figure: ET-Minion core block diagram.

1.1.2. Hardware Threads

1.1.3. Instruction Flow Through the Core

The Front End fetches instructions for both threads from the Neighborhood L0 micro-cache, expands compressed instructions, decodes them, and presents one instruction per cycle to ID, the first stage of the integer pipeline. The integer pipeline executes the instructions in order and passes memory operations to the data cache and vector operations to the VPU. In the opposite direction, the integer pipeline redirects the Front End to a new fetch address after a branch misprediction, exception or other change of control flow, and stalls a thread that must not issue.

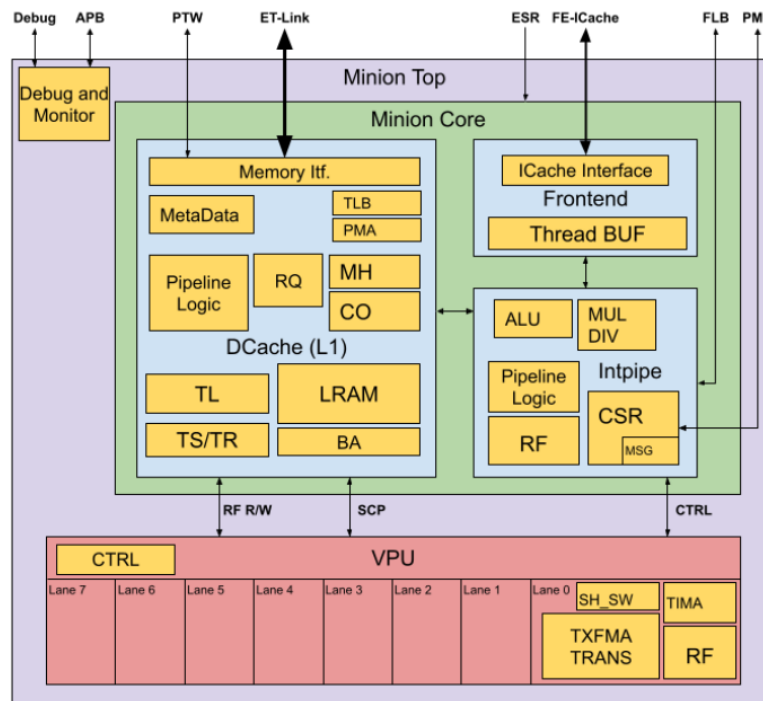


Figure 1. Front End in the ET-Minion core

1.1.4. Interfaces to the Neighborhood

Table: ET-Minion core interface groups.

1.1.5. Clocks, Resets and Power

1.1.6. Debug Support

1.2. Front End

1.2.1. Overview

The Front End is the interface between the Neighborhood L0 micro-cache and the integer pipeline. It holds the program counter of each thread, fetches instructions from the L0 micro-cache, and delivers them, expanded and decoded, to the ID stage of the integer pipeline at a rate of one instruction per cycle. It serves both threads of the core, and restarts fetch at a new address when the integer pipeline redirects a thread.

[Table 1](#) lists the submodules and [Figure 2](#) shows how they are connected.

Table 1. Front End submodules

Submodule	Instances	Function
Thread buffer (<code>frontend_thread_buffer</code>)	2, one per thread	64-byte instruction buffer and the thread's program counter. Issues fetch requests to the L0 micro-cache.
RVC expander (<code>frontend_rvc_expander</code>)	2, one per thread buffer	Expands compressed (RVC) instructions to 32 bits.
Request arbiter (<code>arb_lru_data</code>)	1	2:1 LRU arbiter. Selects the thread whose fetch request is sent to the L0 micro-cache.
Integer decoder (<code>intpipe_decode</code>)	1	Decodes the instruction into integer-pipeline control signals.
VPU decoder (<code>vpu_decoder</code>)	1	Decodes the instruction into VPU control signals.
Instruction FIFO	2, one per thread	Two-entry queue of decoded instructions waiting for the integer pipeline.
Thread scheduler (<code>frontend_thread_sched</code>)	1	Selects the thread whose instruction is presented to the integer pipeline.

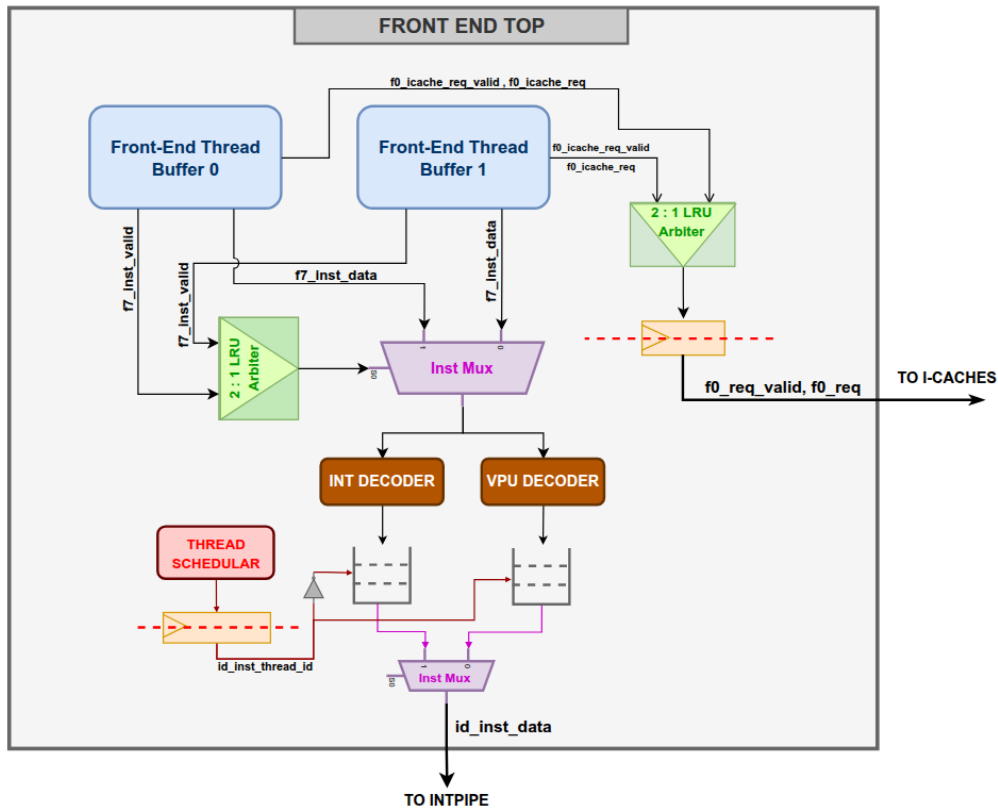


Figure 2. Front End block diagram

1.2.2. Interface

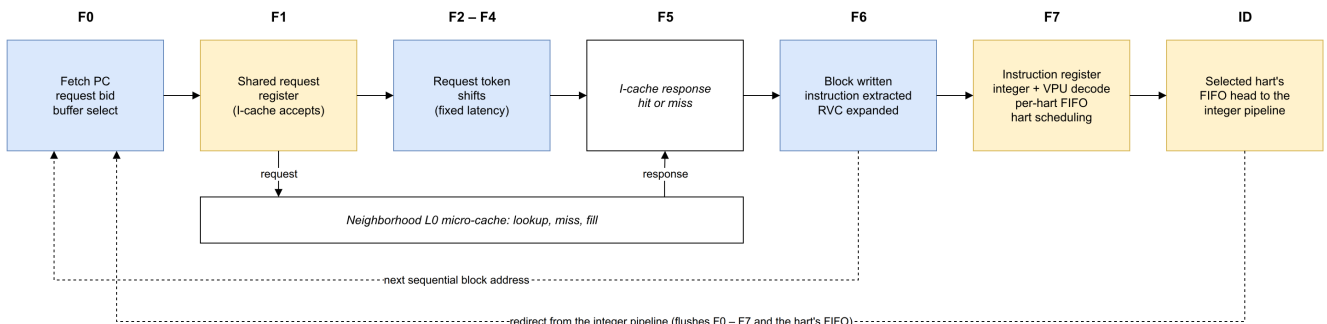
1.2.2.1. I/O Signals

1.2.2.2. Data Structures

1.2.2.3. Configuration Parameters

1.2.3. Pipeline Stages

The Front End pipeline has eight stages, F0 to F7, and delivers instructions to ID, the first stage of the integer pipeline. Stages F1 to F5 overlap the L0 micro-cache pipeline: the fetch request leaves the Front End in F1 and the response returns in F5. Figure 3 shows the stages and Table 2 lists the modules active in each.



Blue: one copy per hart (thread buffer). Yellow: shared by both harts or selects between them.

Figure 3. Front End pipeline

Table 2. Front End pipeline stages

Stage	Module	Operation
F0	Thread buffer, request arbiter	Each thread buffer holds the fetch PC. A thread with a free buffer entry and no request in flight bids for the request port; the 2:1 arbiter grants one thread, which reserves the entry for the returning block. Redirects and thread enables from the integer pipeline are applied in this stage.
F1	F1 request register	The granted request is held in the shared F1 request register and presented to the Neighborhood until accepted. F1 lasts at least two cycles (Section 1.2.4.1).
F2–F4	Thread buffer	The L0 micro-cache processes the request. Each thread buffer advances a token that tracks its own request.
F5	L0 micro-cache	The response returns: a 32-byte block or a miss, with fault and error status.
F6	<code>frontend_top</code> , thread buffer, RVC expander	The response is registered and broadcast to both thread buffers. The owning thread writes the block into the reserved entry, or records the miss. The next instruction is read from the thread buffer and, if compressed, expanded.
F7	Thread buffer, decoders, instruction FIFOs, thread scheduler	Each thread buffer holds one expanded instruction in its F7 register. One thread per cycle is decoded and its instruction written into its FIFO. The thread scheduler selects the thread for ID.
ID	<code>frontend_top</code>	The head of the selected thread’s FIFO is presented to the integer pipeline.

With an L0 hit and no competing requests, the first instruction of a block reaches ID nine cycles after its request leaves F0. The latency and throughput figures are summarized in [Section 1.2.12](#).

[Figure 4](#) shows the Front End pipeline at signal level together with the L0 micro-cache stages.

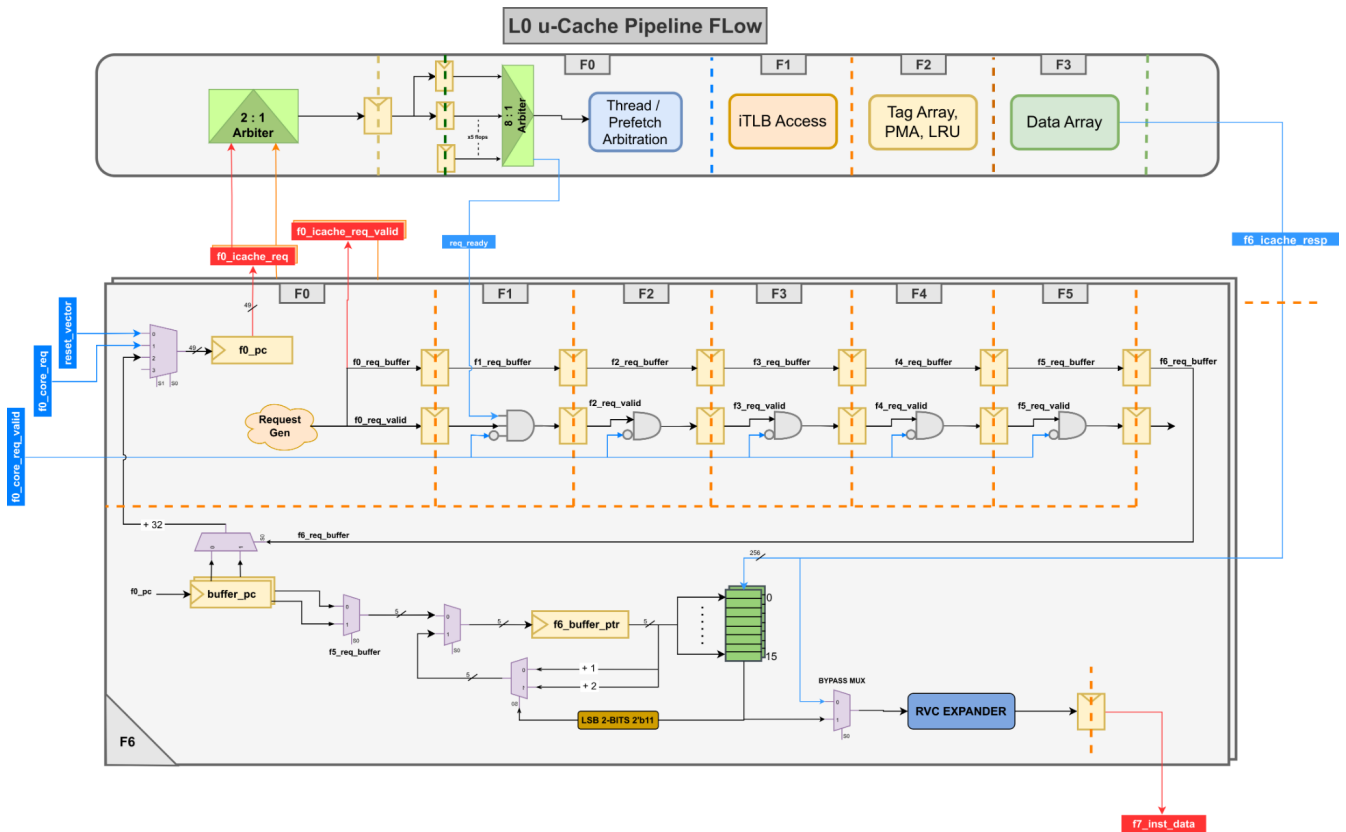


Figure 4. Front End and L0 micro-cache pipeline

1.2.4. Interaction with the L0 Micro-Cache

1.2.4.1. Request Arbitration and Acceptance

1.2.4.2. Response Timing and Ownership

1.2.4.3. Miss Handling and Fill Done

1.2.4.4. Request Cancellation

1.2.5. Thread Buffer

1.2.5.1. Buffer Entries

1.2.5.2. Program Counter

1.2.5.3. Request Generation and Entry Fill

1.2.5.4. Instruction Extraction

1.2.5.5. RVC Expander

1.2.5.6. F7 Register and Fetch Status

1.2.6. Entry Invalidation, Redirect and Thread Start

1.2.6.1. Entry Invalidation

1.2.6.2. Redirect

1.2.6.3. Thread Enable and Reset Vector

1.2.7. Decoders

1.2.8. Instruction FIFO

1.2.9. Thread Scheduler

1.2.10. Halt and Debug Program Buffer

1.2.11. Clock Gating

1.2.12. Performance

1.2.13. Known Limitations and Verification Notes

1.3. Integer Pipeline

1.3.1. Overview

Figure: Integer pipeline block diagram.

1.3.2. Pipeline Stages

Figure: Integer pipeline stages (ID, EX, TAG, MEM, WB).

1.3.3. Interfaces

Table: Integer pipeline interface signals.

1.3.4. Register File and Operand Read

1.3.5. Immediate Generation and ALU

1.3.6. Multiply/Divide Unit

1.3.7. Scoreboards and Hazards

1.3.7.1. Integer Scoreboard

1.3.7.2. FP Scoreboard

1.3.7.3. Mask Scoreboard

1.3.8. Hart Interleaving and Stalls

1.3.9. Branches, Redirects and Replay

1.3.10. Interface to the Data Cache

1.3.11. Interface to the VPU

1.3.12. CSRs and Privilege Modes

Table: CSR groups.

1.3.13. Exceptions and Interrupts

1.3.14. Debug and Breakpoints

1.3.15. Performance Counters

1.3.16. Verification Notes

1.4. Data Cache

1.4.1. Overview

Figure: Data cache block diagram.

1.4.2. Organization and Operating Modes

Table: Data cache operating modes.

1.4.3. Interfaces

Table: Data cache interface signals.

1.4.4. Request Pipeline

Figure: Data cache request pipeline.

1.4.5. Address Translation and PMA Checks

1.4.6. Tag, Data and LRU Arrays

1.4.7. Miss Handling

1.4.8. Replay Queue

1.4.9. Writeback and Store Merge

1.4.10. Atomic Operations

1.4.11. Cache-Management Operations

1.4.12. Scratchpad Control

1.4.13. Tensor Load and Reduce

1.4.14. Interface to the Neighborhood

1.4.15. Errors

1.4.16. Performance

1.4.17. Verification Notes

1.5. Vector Processing Unit

1.5.1. Overview

Figure: VPU block diagram.

1.5.2. Interfaces

Table: VPU interface signals.

1.5.3. Control and Micro-Instruction Decode

1.5.4. Register Files

1.5.5. Lanes

Figure: VPU lane.

1.5.6. Mask Unit

1.5.7. Bypass Network

1.5.8. Shift and Swizzle Unit

1.5.9. TxFMA Unit

1.5.10. Integer Multiply-Accumulate Unit

1.5.11. Transcendental Unit

1.5.12. Tensor Operations

1.5.12.1. Tensor FMA

1.5.12.2. Tensor Quantization

1.5.12.3. Tensor Reduce

1.5.13. Interfaces to the Integer Pipeline and Data Cache

1.5.14. Performance

1.5.15. Verification Notes